Gerard López Golfe

Caso de estudio | SecretarIA, un asistente RAG self-hosted en producción

Vilamarxant (Valencia), España | linkedin.com/in/gerard-skt
Código y registro de decisiones: github.com/SureKT/secretaria-rag

EL PROBLEMA

Las secretarías de dos centros públicos de FP de Valencia (CIPFP Ausiàs March y CIPFP Mislata) responden todo el año las mismas preguntas: ciclos, matrícula, becas, horarios y trámites. Querían un asistente en su web con una condición innegociable: ninguna consulta de alumnos ni personal podía salir del edificio.

LO QUE CONSTRUÍ (EN SOLITARIO, MAR – JUL 2026)

- Requisitos con los centros, ingesta, recuperación, agente, suite de evaluación y despliegue.
- Widget en la web → servicio FastAPI que envía la respuesta token a token con server-sent events.
- Recuperación: embeddings mxbai-embed-large en Qdrant, chunking jerárquico y un filtro por centro en cada búsqueda (multi-tenant), más el contenido común a los dos centros.
- Generación con un LLM local en Ollama, sobre el NVIDIA DGX Spark del instituto. Memoria de conversación en PostgreSQL; Docker Compose y nginx. 25 tests con dobles de Qdrant, Ollama y PostgreSQL.

Resultado: las llamadas a secretaría bajaron a una cuarta parte en pocas semanas.

TRES DECISIONES

- **Sustituí mi propia primera versión.** Funcionaba sobre n8n, que solo devuelve respuestas completas, y la gente interpretaba varios segundos de silencio como un bot roto. Reescribí el backend en FastAPI para enviar tokens en streaming: mismo modelo, mismas respuestas, y el problema desapareció.
- **Las respuestas erróneas no eran culpa del modelo.** Las preguntas sobre un ciclo devolvían el contenido de otro. Construí una auditoría de recuperación (fragmentos recuperados y su puntuación, sin LLM) para separar fallos de búsqueda de fallos del modelo. La causa: la búsqueda de texto no manejaba las tildes. La recuperación pasó a embeddings.
- **Un índice para dos centros.** El contenido común se guarda una vez; el aislamiento depende de un filtro por centro que se construye en una sola función y que comprueban los tests. Las opciones descartadas están en un registro de decisiones de arquitectura (ADR).

EVALUACIÓN

El modelo de producción se eligió con una suite de 36 casos, no por impresión: preguntas por familia profesional, horarios y trámites; **casos negativos**, en los que lo correcto es no responder y redirigir; y **cadenas multi-turno** que comprueban la memoria de conversación. Cada caso registra acierto y latencia. La suite detectó también falsos positivos en sus propias comprobaciones, que corregí.

LÍMITES Y SIGUIENTES PASOS

- Aún sin reranking: una pasada con cross-encoder tras la recuperación es la mejora evidente.
- Comprobaciones por palabras clave, no evaluadas por un modelo, y sin feedback del uso real.

Stack: Python, FastAPI, Qdrant, Ollama (mistral-small3.2), mxbai-embed-large, PostgreSQL, Docker Compose, nginx, n8n (primera versión).